

Playwright with Python & Pytest — Complete Study Notes

8-Week Comprehensive Course • Detailed Edition

50+ Annotated Examples • Full Code Snippets • Deep-Dive Notes

Week 1	Python Revision, Manual vs Automation, Playwright Intro, Setup
Week 2	Browser Commands, HTML Basics, Locators, Web Elements
Week 3	Dropdowns, Checkboxes, Actions, Tables, Frames
Week 4	Windows, Alerts, Excel, Waits, GitHub & Jenkins
Week 5	Pytest Intro, Annotations, Assertions (Hard & Soft)
Week 6	Fixtures, Coverage, Test Priority & Ordering
Week 7	Timeout, Parallel Testing, Parameterization
Week 8	Cross-Browser, Reports, Allure, GitHub Copilot AI

Practice Site: <https://www.saucedemo.com> — Use this throughout all 8 weeks!

WEEK 1 — Python Revision, Manual vs Automation & Playwright Setup

1. Python Revision

Python is the programming language used to write Playwright automation scripts. Before diving into browser automation, it is essential to be comfortable with Python's core building blocks. Below is a detailed walkthrough of everything you need.

Variables & Data Types

A **variable** is a named container that holds a value. Python is dynamically typed — you don't need to declare the type; Python figures it out automatically. The four most common types are: **str** (text), **int** (whole number), **float** (decimal number), and **bool** (True/False).

```
name = 'John' # str – text in single or double quotes
age = 25 # int – whole number
price = 99.5 # float – decimal
is_active = True # bool – True or False

# Type checking
print(type(name)) # <class 'str'>
print(type(age)) # <class 'int'>

# String formatting (f-strings) – very useful in test messages
print(f'User {name} is {age} years old') # User John is 25 years old
```

Lists & Dictionaries

A **list** is an ordered collection of items (like a shopping list). You access items by their **index**, starting at 0. A **dictionary** stores key-value pairs — like a real dictionary where a word (key) maps to its meaning (value). Both are extremely common in test data management.

```
# List – ordered, changeable
browsers = ['chrome', 'firefox', 'safari']
print(browsers[0]) # chrome (first item = index 0)
print(browsers[-1]) # safari (last item = index -1)
browsers.append('edge') # add item to end
print(len(browsers)) # 4

# Dictionary – key:value pairs
user = {'name': 'Alice', 'age': 30, 'role': 'admin'}
print(user['name']) # Alice
user['email'] = 'a@b.com' # add new key
print(user.keys()) # dict_keys(['name', 'age', 'role', 'email'])
```

Functions

A **function** is a reusable block of code. You define it once with **def** and call it many times. Functions accept **parameters** (inputs) and can **return** a result. In automation, functions help avoid code duplication — e.g., a login() function used across many test files.

```
def greet(name):
    return f'Hello, {name}!'

print(greet('Alice')) # Hello, Alice!

# Function with default parameter value
def launch_browser(browser='chromium', headless=False):
    print(f'Launching {browser}, headless={headless}')

launch_browser() # chromium, headless=False
launch_browser('firefox', True) # firefox, headless=True
```

Loops

Loops let you repeat actions. The **for** loop iterates over a sequence (list, range, etc.). The **while** loop runs as long as a condition is True. In Playwright, loops are used to iterate over table rows, list items, or test data sets.

```
# for loop with range
for i in range(5): # 0, 1, 2, 3, 4
    print(i)

# for loop over a list
urls = ['https://google.com', 'https://bing.com']
for url in urls:
    print(f'Testing: {url}')

# while loop
retry = 0
while retry < 3:
    print(f'Attempt {retry + 1}')
    retry += 1
```

Classes & Objects

A **class** is a blueprint for creating objects. An **object** is an instance of that class. The `__init__` method runs automatically when you create an object — it's the constructor. In automation, the Page Object Model (POM) uses classes to represent web pages as objects.

```
class LoginPage:
    def __init__(self, page):
        self.page = page
        self.url = 'https://example.com/login'

    def open(self):
        self.page.goto(self.url)

    def login(self, username, password):
        self.page.locator('#user').fill(username)
        self.page.locator('#pass').fill(password)
        self.page.locator('#submit').click()

# Usage:
# lp = LoginPage(page)
# lp.open()
# lp.login('admin', 'pass123')
```

■ *Python Full Course for Beginners — Programming with Mosh | youtube.com/watch?v=_uQrJ0TkZlc*

2. Manual Testing vs Automation Testing

Testing ensures software works correctly before it reaches end users. There are two fundamental approaches: doing it manually, or writing code to do it automatically.

Feature	Manual Testing	Automation Testing
	Human tester clicks/checks manually	A script/tool does it automatically
Speed	Slow — depends on human pace	Very fast — runs in seconds
	Low — just need a tester	Higher — need to write & maintain scripts
Long-term Cost	Expensive if run repeatedly	Cheap — scripts can run 1000s of times
	Errors due to fatigue/oversight	100% consistent on every run
Best Use	Exploratory, UI/UX, one-time checks	Regression, repeated, data-driven tests
	Slow — hours to days	Instant — results in minutes

■ **TIP** Simple Example: Manually — you open the browser, go to google.com, type a search, and verify results yourself. With Playwright — the script does ALL of that in 2 seconds, every time, without errors.

■ *Manual vs Automation Testing — Software Testing Mentor | youtube.com/watch?v=Mp28br_bP8E*

3. Introduction to Playwright

Playwright is an open-source browser automation library created by **Microsoft** in 2020. It lets you write code that controls web browsers — just like a human user would, but much faster and more reliably. Playwright has quickly become the industry standard for end-to-end (E2E) web testing.

Key features of Playwright:

- **Multi-browser:** Works with Chromium (Chrome/Edge), Firefox, and WebKit (Safari) — all with the same API.
- **Multi-language:** Supports Python, JavaScript/TypeScript, Java, and C#.
- **Auto-wait:** Automatically waits for elements to be ready before interacting — no manual `sleep()` calls needed.
- **Headless mode:** Runs browser invisibly (no UI) for faster CI/CD pipeline execution.
- **Network interception:** Can mock API responses, block requests, or spy on network calls.
- **Screenshots & Videos:** Can capture screenshots and record videos of test runs automatically.
- **Parallel execution:** Runs multiple tests simultaneously for speed.
- **Codegen:** Can record your manual browser actions and generate test code automatically.

■ **TIP** Think of Playwright as a ROBOT that controls the browser: it opens tabs, navigates URLs, finds elements, clicks buttons, fills forms, and checks results — all without any human interaction.

■ *Playwright Python Tutorial for Beginners | youtube.com/watch?v=S8PHsJmhFcA*

4. Installation & Setup

Installing Playwright requires two steps: (1) install the Python library via pip, and (2) download the browser binaries. Playwright ships its own browser versions to ensure consistent, reproducible test behaviour.

```
# Step 1 – Install the Playwright Python package
pip install playwright

# Step 2 – Download browser binaries (Chrome, Firefox, Safari)
playwright install

# Step 3 – Verify installation with a quick test
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False) # headless=False shows the browser
    page = browser.new_page()
    page.goto('https://google.com')
    print(page.title()) # Google
    browser.close()
```

■ **TIP** If you see 'Google' printed, your setup works! Use headless=True in production/CI — it's faster. Use headless=False while learning so you can SEE what the browser is doing.

■ *Playwright Python Installation — Naveen AutomationLabs | [youtube.com/watch?v=mFTHqEkGq5s](https://www.youtube.com/watch?v=mFTHqEkGq5s)*

WEEK 2 — Browser Commands, HTML Basics, Locators & Web Elements

1. Basic Playwright Commands

Every Playwright script follows the same pattern: create a playwright context, launch a browser, open a page, interact with it, and close the browser. The **with** statement ensures the browser is properly closed even if an error occurs.

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False) # Open browser window
    page = browser.new_page() # Open a new tab

    page.goto('https://example.com') # Navigate to a URL
    print(page.title()) # Get the <title> tag text
    print(page.url) # Get current URL

    page.screenshot(path='screenshot.png') # Save a PNG screenshot
    browser.close() # Always close the browser
```

2. Browser Configuration Options

Playwright gives you fine-grained control over how the browser is launched and configured. These options affect every page opened in that browser session.

```
browser = p.chromium.launch(
    headless = False, # True = no UI window (faster for CI)
    slow_mo = 1000, # Pause 1 sec between each action (great for debugging)
    args = ['--start-maximized'] # Pass Chrome flags
)

# Context options – apply to all pages in this context
context = browser.new_context(
    viewport = {'width': 1920, 'height': 1080},
    ignore_https_errors = True, # Ignore SSL errors
    record_video_dir = './videos' # Record video of every test
)
page = context.new_page()

# Navigation commands
page.go_back() # Browser back button
page.go_forward() # Browser forward button
page.reload() # F5 / refresh
```

■ **TIP** Use `slow_mo=500` when debugging — it slows every action so you can visually follow what Playwright is doing.

3. Launch in Different Browsers

One of Playwright's greatest strengths is the ability to run the same test on multiple browsers with no code changes — just swap the launcher.

```

with sync_playwright() as p:
    # Chromium — covers Chrome and Edge
    browser = p.chromium.launch(headless=False)

    # Firefox — Mozilla Firefox
    browser = p.firefox.launch(headless=False)

    # WebKit — Apple Safari engine
    browser = p.webkit.launch(headless=False)

    page = browser.new_page()
    page.goto('https://example.com')
    print(f'Title: {page.title()}')
    browser.close()

```

■ *Playwright Basic Commands* | [youtube.com/watch?v=ySbPsJUTRwc](https://www.youtube.com/watch?v=ySbPsJUTRwc)

4. Basic HTML Concepts for Testers

HTML (HyperText Markup Language) is the skeleton of every web page. As an automation tester, you don't need to BUILD websites — but you need to READ HTML to understand how to find and interact with elements. Every interactive element (button, input, link) has HTML **attributes** that you use as locators.

```

<!-- A typical login form in HTML -->
<form>
<input type='text' id='username' name='user' placeholder='Enter email' />
<input type='password' id='password' name='pass' placeholder='Password' />
<button type='submit' id='loginBtn' class='btn primary'>Login</button>
<a href='/forgot'>Forgot password?</a>
</form>

```

Attribute	What it is	Example
	Unique identifier — best for locating elements	id="loginBtn"
class	Groups elements for styling; multiple elements can share	class="btn primary"
	Used in form submissions to name the field	name="username"
href	The URL a link points to	href="/home"
	Input field type (text, password, checkbox, etc.)	type="password"
placeholder	Grey hint text shown before the user types	placeholder="Enter email"
	Current value of an input or a button's label	value="Submit"

5. How to Inspect Elements

Browser DevTools is the tester's most important tool. It lets you see the HTML behind any visible element on the page. This is how you find the right id, class, or XPath to use as a locator.

- **Step 1:** Open any website in Chrome or Edge.
- **Step 2:** Right-click on the element you want to automate.
- **Step 3:** Click **Inspect** — the DevTools panel opens on the right.
- **Step 4:** The highlighted HTML shows the element's attributes.
- **Step 5:** Look for **id** first, then **class**, then **name**.
- **Shortcut:** Press F12 to open DevTools directly.

■ **TIP** Right-click in the Elements tab → 'Copy' → 'Copy selector' to get a CSS selector automatically. Or 'Copy XPath' for an XPath locator.

■ *How to Inspect Elements — Chrome DevTools Basics | youtube.com/watch?v=x4_i1MOBdq*

6. Locator Techniques (Priority Order)

A **locator** is how Playwright finds an element on the page. Choosing the right locator makes your tests more stable and readable. Playwright recommends a specific priority order — use the most reliable locator first.

```
# BEST — Playwright recommended locators (semantic & stable)
page.get_by_role('button', name='Submit') # By ARIA role + accessible name
page.get_by_label('Username') # By the <label> text for an input
page.get_by_placeholder('Enter your email') # By placeholder text
page.get_by_text('Welcome back!') # By visible text content
page.get_by_test_id('submit-btn') # By data-testid attribute

# GOOD — CSS selectors (fast, widely used)
page.locator('#username') # By id (# prefix)
page.locator('.btn-login') # By class (. prefix)
page.locator('input[type="text"]') # By attribute + value
page.locator('div.container > button') # Nested CSS selector

# LAST RESORT — XPath (brittle, harder to read)
page.locator('//input[@id="username"]') # Absolute XPath
page.locator('//button[text()="Login"]') # XPath by text
```

■ **TIP** Priority: `get_by_role` > `get_by_label` > `get_by_placeholder` > `get_by_text` > CSS `#id` > CSS `.class` > XPath. Avoid XPath unless nothing else works — it breaks easily when the page structure changes.

■ *Playwright Locators Tutorial — Naveen AutomationLabs | youtube.com/watch?v=rqTTVGw2T8Q*

7. Web Element Commands

Once you've located an element, you interact with it using these commands. All commands include Playwright's built-in auto-wait — Playwright waits for the element to be visible and actionable before performing the action.


```
loc = page.locator('#username') # Store locator in a variable

# Input actions
loc.fill('admin') # Clear field then type text (recommended)
loc.type('admin', delay=100) # Type character by character (simulates human)
loc.clear() # Clear the field
loc.press('Enter') # Press a keyboard key
loc.press('Tab') # Tab to next field

# Click actions
page.locator('#submit').click() # Left click
page.locator('#item').dbl_click() # Double click
page.locator('#menu').click(button='right') # Right click

# Read data from elements
text = page.locator('h1').inner_text() # Visible text
html = page.locator('div').inner_html() # Inner HTML string
value = page.locator('#email').get_attribute('placeholder') # Any attribute
```

WEEK 3 — Dropdowns, Checkboxes, Actions, Tables & Frames

1. Handling Dropdowns (select Element)

An HTML **select** element creates a dropdown menu. Playwright provides the **select_option()** method with three ways to pick a value: by the visible text (label), by the underlying value attribute, or by its position (index).

```
# Select by visible text (what the user sees)
page.locator('#country').select_option(label='India')

# Select by option value (HTML value attribute)
page.locator('#country').select_option(value='IN')

# Select by position (0 = first option)
page.locator('#country').select_option(index=2)

# Read the currently selected value
selected = page.locator('#country').input_value()
print(f'Selected: {selected}') # e.g., 'IN'

# Multi-select dropdown (select multiple options at once)
page.locator('#colours').select_option(['red', 'blue'])
```

■ **TIP** Always prefer `select_option(label=...)` — it uses what the user sees, making tests more readable. Use `value=` only when labels contain special characters or duplicates.

■ Playwright Dropdown Handling | youtube.com/watch?v=7JsS5xFkWDk

2. Checkboxes & Radio Buttons

Checkboxes allow users to select one or more options independently. Radio buttons allow only ONE option from a group to be selected at a time. Playwright provides dedicated methods that are more reliable than a plain `click()`.

[illegible]

3. State Check Methods: `is_checked()`, `is_enabled()`, `is_visible()`

These three methods let you inspect the current state of any element without interacting with it. They are essential for writing conditional logic and smart assertions in your tests.

```
# is_checked() – True if checkbox/radio is selected
checked = page.locator('#agree').is_checked()
print(f'Terms accepted: {checked}') # True or False

# is_enabled() – True if element can be interacted with (not greyed out)
enabled = page.locator('#submit').is_enabled()
if enabled:
    page.locator('#submit').click()
else:
    print('Submit button is disabled – form is not ready')

# is_visible() – True if element is currently shown on screen
visible = page.locator('#popup').is_visible()
if visible:
    page.locator('#close-btn').click() # Close the popup first

# is_hidden() – opposite of is_visible()
hidden = page.locator('#loader').is_hidden()
```

■ **TIP** Always guard clicks with `is_enabled()` checks in tests where button states depend on previous form fields — e.g., 'Submit' is disabled until all required fields are filled.

4. Mouse & Keyboard Actions

Playwright can simulate any mouse action or keyboard shortcut that a real user would perform. These are especially important for testing dropdowns, context menus, drag-and-drop features, and keyboard shortcuts.

```
# Mouse hover – triggers CSS :hover effects, reveals dropdown menus
page.locator('#navbar-menu').hover()

# Double click – selects text or opens items in file managers
page.locator('#file-item').dbl_click()

# Right click – opens context menu
page.locator('#image').click(button='right')

# Drag and drop – drag source to target element
page.locator('#drag-item').drag_to(page.locator('#drop-zone'))

# Keyboard shortcuts
page.keyboard.press('Control+A') # Select all text
page.keyboard.press('Control+C') # Copy
page.keyboard.press('Control+V') # Paste
page.keyboard.press('Escape') # Close modal/dropdown
page.keyboard.press('F5') # Refresh page

# Type text with realistic delay (simulates human typing speed)
page.locator('#search').type('playwright python', delay=80)

# Scroll page
page.mouse.wheel(0, 500) # Scroll down 500 pixels
```

■ *Playwright Mouse and Keyboard Actions | [youtube.com/watch?v=1k8EKdSkFYI](https://www.youtube.com/watch?v=1k8EKdSkFYI)*

5. Handling Tables

Web tables display data in rows and columns. Playwright handles tables by locating the container, counting rows, and then accessing individual cells. This pattern works for any HTML table regardless of size.

```
# Get all rows in the table body (skip the header row)
rows = page.locator('table tbody tr')
count = rows.count()
print(f'Total rows: {count}')
```

```
# Iterate over every row
for i in range(count):
    row = rows.nth(i) # Get row at position i
    cells = row.locator('td') # Get all cells in this row
    name = cells.nth(0).inner_text() # First column
    age = cells.nth(1).inner_text() # Second column
    print(f'Row {i+1}: Name={name}, Age={age}')
```

```
# Find a specific value in the table
all_names = [rows.nth(i).locator('td').nth(0).inner_text()
              for i in range(count)]
print('John' in all_names) # True if John exists in the table
```

■ *Playwright Table Handling | [youtube.com/watch?v=SFUHGTgSwwI](https://www.youtube.com/watch?v=SFUHGTgSwwI)*

6. Handling Frames (iFrames)

An **iFrame** is a web page embedded inside another web page. Elements inside an iFrame are completely separate from the main page — you cannot locate them with a normal `page.locator()`. You must first switch INTO the frame's context, then interact normally.

```
# Method 1: frame_locator() – modern Playwright approach (recommended)
frame = page.frame_locator('#myIframe') # Locate the iframe by CSS
frame.locator('#username').fill('admin') # Interact inside iframe
frame.locator('#login').click()
```

```
# Method 2: frame() by name attribute
frame = page.frame(name='loginFrame') # <iframe name='loginFrame'>
frame.locator('#user').fill('admin')
```

```
# Method 3: frame() by URL pattern
frame = page.frame(url='**/payment.html')
```

```
# Nested iframes (iframe inside an iframe)
outer_frame = page.frame_locator('#outer')
inner_frame = outer_frame.frame_locator('#inner')
inner_frame.locator('#cvv').fill('123')
```

■ **TIP** Common in payment gateways (PayPal, Stripe) and Google Maps embeds. If your locator can't find an element, check if it's inside an iFrame first!

■ *Playwright iFrame Handling | [youtube.com/watch?v=JW8OoUTbWfo](https://www.youtube.com/watch?v=JW8OoUTbWfo)*

WEEK 4 — Windows, Alerts, Excel, Waits, GitHub & Jenkins

1. Multiple Window / Tab Handling

Some web actions (clicking a link with `target='_blank'`, or JavaScript `window.open()`) open a NEW browser tab or window. Playwright captures this event using **`expect_popup()`**, which waits for the new page to open and returns it to you.

```
# Wait for a new tab/window to open after clicking a button
with page.expect_popup() as popup_info:
    page.locator('#open-new-tab').click() # This triggers a new tab

new_page = popup_info.value # The new tab's Page object
print(f'New tab URL: {new_page.url}')

# Work in the new tab
new_page.locator('#username').fill('admin')
new_page.locator('#submit').click()

# Switch back to original tab
new_page.close() # Close new tab
page.bring_to_front() # Make original tab active

# Access all open pages (tabs) in a context
all_pages = context.pages
print(f'Open tabs: {len(all_pages)}')
```

■ *Playwright Multiple Windows / New Tab Handling | youtube.com/watch?v=4sSjqKs3M4*

2. Handling JavaScript Alerts, Confirms & Prompts

JavaScript dialogs (alert, confirm, prompt) are browser-native popups that overlay the page. Unlike normal HTML elements, you cannot locate them with CSS or XPath. Playwright handles them through the **`dialog`** event — you register a handler *before* the action that triggers the dialog.

```

# Alert – has only OK button
page.on('dialog', lambda d: d.accept()) # Register BEFORE clicking
page.locator('#alert-btn').click() # Now click

# Confirm – has OK and Cancel buttons
page.on('dialog', lambda d: d.accept()) # Click OK
page.on('dialog', lambda d: d.dismiss()) # Click Cancel

# Prompt – has a text input + OK/Cancel
page.on('dialog', lambda d: d.accept('John')) # Type 'John' and click OK

# Read the dialog message first, then act on it
def smart_dialog_handler(dialog):
    print(f'Dialog type : {dialog.type}') # alert/confirm/prompt
    print(f'Dialog message: {dialog.message}')
    if 'delete' in dialog.message.lower():
        dialog.dismiss() # Cancel destructive actions
    else:
        dialog.accept()

page.on('dialog', smart_dialog_handler)
page.locator('#confirm-btn').click()

```

■ *Playwright Alert Handling | youtube.com/watch?v=QgPxGKWbDiY*

3. Reading Excel Files for Test Data (openpyxl)

Real-world tests often need to run with many sets of data (e.g., 50 users, 100 products). Storing test data in Excel spreadsheets and reading them with **openpyxl** is a powerful pattern called **Data-Driven Testing**.

```

pip install openpyxl

import openpyxl

wb = openpyxl.load_workbook('testdata.xlsx') # Open workbook
ws = wb.active # Use first sheet
# ws = wb['Sheet2'] # Or pick by name

# Read a specific cell
print(ws['A1'].value) # Row 1, Column A
print(ws.cell(row=2, column=3).value) # Row 2, Column C

# Read all rows
for row in ws.iter_rows(values_only=True):
    print(row) # Each row as a tuple: ('admin', 'pass123', 'welcome')

# Practical: drive a login test from Excel
for row in ws.iter_rows(min_row=2, values_only=True): # Skip header row
    username, password, expected = row
    page.locator('#user').fill(str(username))
    page.locator('#pass').fill(str(password))
    page.locator('#login').click()
    assert expected in page.inner_text('body')
    page.go_back()

```

■ *openpyxl Tutorial | youtube.com/watch?v=u3Ue-fqVBOW*

4. Waits — Dealing with Slow or Dynamic Pages

Modern web apps load content dynamically (AJAX, React, Vue). An element may not exist immediately when the page loads. Playwright has **built-in auto-wait** — it waits up to 30 seconds by default for elements to be ready. But sometimes you need explicit waits for special cases.

```
# Wait for a specific element to appear in the DOM
page.wait_for_selector('#result-table') # Waits until element exists

# Wait for a fixed duration (avoid this – use conditional waits instead)
page.wait_for_timeout(3000) # Wait 3000ms = 3 seconds

# Wait for URL to change (e.g., after form submission redirects)
page.wait_for_url('https://example.com/dashboard')
page.wait_for_url('**/success**') # Wildcard URL matching

# Wait for element state changes
page.locator('#spinner').wait_for(state='hidden') # Wait until gone
page.locator('#results').wait_for(state='visible') # Wait until shown

# Wait for network response (useful for API calls)
with page.expect_response('**/api/login') as resp_info:
    page.locator('#submit').click()
    response = resp_info.value
    print(response.status) # 200

# Change the default timeout (milliseconds)
page.set_default_timeout(60000) # 60 seconds for slow pages
```

■ **TIP** Never use `wait_for_timeout()` as your primary wait strategy — it makes tests slow and fragile. Always prefer waiting for specific elements or conditions.

5. GitHub — Version Control for Your Tests

GitHub is where you store and manage your test code. Version control lets you track changes, collaborate with teammates, and roll back if something breaks. Every professional automation engineer uses Git daily.

```
# Initialise a new local repository in your project folder
git init

# See which files have changed
git status

# Stage all changed files for commit
git add .

# Save a snapshot with a descriptive message
git commit -m 'Add login test with parameterization'

# Connect to a remote GitHub repository
git remote add origin https://github.com/yourname/repo.git

# Push your commits to GitHub
git push -u origin main

# Pull latest changes from remote (team collaboration)
git pull origin main
```

6. Jenkins — Continuous Integration (CI/CD)

Jenkins is an open-source automation server that can run your Playwright tests automatically — triggered by a code push, on a schedule, or manually. This is called **Continuous Integration** — ensuring tests always run against the latest code.

- Download and install Jenkins from jenkins.io
- Open Jenkins in your browser at **`http://localhost:8080`**
- Create a New Job → choose **Freestyle Project**
- In Source Code Management → add your GitHub repo URL
- In Build Triggers → enable **GitHub hook trigger for GITScm polling**
- In Build Steps → add: **`pytest tests/ --html=report.html -v`**
- Save and click **Build Now** — Jenkins runs your tests!
- View test results and HTML report in the build artifacts.

■ *GitHub: [youtube.com/watch?v=RGQj5yH7evk](https://www.youtube.com/watch?v=RGQj5yH7evk) | Jenkins: [youtube.com/watch?v=LFDnKPOTg](https://www.youtube.com/watch?v=LFDnKPOTg)*

WEEK 5 — Introduction to Pytest & Assertions

1. What is Pytest?

Pytest is the most popular Python testing framework. It provides a clean structure for writing, organising, and running tests. Pytest discovers test files and functions automatically based on naming conventions, runs them, and produces a detailed pass/fail report.

- **Simple syntax:** Just write plain functions starting with test_
- **Auto-discovery:** Finds test files/functions without configuration
- **Rich assertions:** Normal Python assert gives detailed failure messages
- **Fixtures:** Reusable setup/teardown code (Week 6)
- **Plugins:** 1,000+ plugins — HTML reports, coverage, parallel runs, etc.
- **Parametrize:** Run same test with multiple data sets (Week 7)

2. Pytest Installation

```
pip install pytest # Core pytest
pip install pytest-playwright # Playwright integration for pytest

pytest --version # Verify: pytest 8.x.x

# Run tests
pytest # Run ALL tests in current folder
pytest test_login.py # Run one specific file
pytest -v # Verbose – show each test name
pytest -v -s # Also show print() output
pytest -k 'test_login' # Run tests whose name contains 'test_login'
pytest --tb=short # Shorter traceback on failures
```

3. Pytest Naming Rules & Structure

Rule	Convention	Example
	Must start with test_ OR end with _test	test_login.py / login_test.py
Test functions	Must start with test_	def test_login():
	Must start with Test (no __init__)	class TestLogin:
Config file	pytest.ini or conftest.py in root	pytest.ini

```
# test_login.py – a complete test file example
from playwright.sync_api import Page

def test_valid_login(page: Page):
    '''Test that valid credentials log in successfully.'''
    page.goto('https://www.saucedemo.com')
    page.locator('#user-name').fill('standard_user')
    page.locator('#password').fill('secret_sauce')
    page.locator('#login-button').click()
    assert page.url == 'https://www.saucedemo.com/inventory.html'
    assert page.locator('.inventory_list').is_visible()

def test_page_title(page: Page):
    page.goto('https://google.com')
    assert page.title() == 'Google'
```

■ *Pytest Full Tutorial — Corey Schafer | youtube.com/watch?v=byaxg00Gf9I*

4. Hard Assertions vs Soft Assertions

An **assertion** checks if an expected condition is true. If the check fails, the test fails. There are two behaviours when an assertion fails: **hard** (stop immediately) and **soft** (continue and collect all failures).

Hard Assertions — Default Behaviour

Python's built-in **assert** is a hard assertion. When it fails, execution stops at that line. Any assertions after the failed one are NEVER checked. Good for critical checks where there's no point continuing.

```
def test_hard_assertions():
    title = 'Google'
    length = len(title)

    assert title == 'Google', 'Title should be Google' # PASS
    assert length == 10, 'Length should be 10' # FAIL – stops here
    assert title.startswith('G'), 'Should start with G' # NEVER RUNS

# Output: AssertionError: Length should be 10
# (You don't know the 3rd assertion would also fail)
```

Soft Assertions — pytest-check

Soft assertions let ALL checks run regardless of individual failures. All failures are collected and reported together at the end. This is useful for UI testing where you want to check multiple properties of a page in one test run.

```
pip install pytest-check

import pytest_check as check

def test_soft_assertions(page):
    page.goto('https://www.saucedemo.com')
    page.locator('#user-name').fill('standard_user')
    page.locator('#password').fill('secret_sauce')
    page.locator('#login-button').click()

    check.equal(page.title(), 'Swag Labs') # PASS
    check.is_true(page.locator('.inventory_list').is_visible()) # PASS
    check.equal(page.url, 'https://wrong-url.com') # FAIL – but continues!
    check.greater(page.locator('.inventory_item').count(), 5) # PASS

# End of test: ALL failures reported together
```

■ **TIP** Use hard assertions for critical flows (login redirect). Use soft assertions when checking multiple UI elements on the same page — so one failure doesn't hide others.

■ *Hard vs Soft Assertion in Pytest | [youtube.com/watch?v=_Vb9Jd1O4vE](https://www.youtube.com/watch?v=_Vb9Jd1O4vE)*

WEEK 6 — Fixtures, Coverage & Test Ordering

1. Fixtures — Reusable Setup & Teardown

A **fixture** is a function decorated with `@pytest.fixture` that provides setup code to tests that request it. Instead of repeating browser setup in every test, write it once as a fixture — all tests use it automatically by including its name as a parameter.

The key keyword is **yield** — everything before yield is **setup**, everything after yield is **teardown** (cleanup). Pytest guarantees cleanup runs even if the test fails.

```
# conftest.py – fixtures defined here are available to ALL test files
import pytest
from playwright.sync_api import sync_playwright

@pytest.fixture
def page():
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=False)
        context = browser.new_context()
        page = context.new_page()
        yield page # ← TEST RUNS HERE – page is passed to test
        browser.close() # ← TEARDOWN – always runs after test

# test_homepage.py – uses the fixture
def test_title(page): # 'page' fixture injected automatically
    page.goto('https://www.saucedemo.com')
    assert page.title() == 'Swag Labs'

def test_login_visible(page):
    page.goto('https://www.saucedemo.com')
    assert page.locator('#login-button').is_visible()
```

Fixture Scope — How Often Does the Fixture Run?

Scope	Runs Once Per...	Best For
	Every single test function	Tests needing a fresh browser state
class	Each test class	Grouping related tests
	Each .py test file	Tests that share a logged-in session
session	The entire pytest run	Expensive setup like DB connections

```
@pytest.fixture(scope='module') # Browser opened once for entire file
def browser_session():
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=False)
        yield browser
        browser.close()
```

■ *Pytest Fixtures Tutorial — NeuralNine | [youtube.com/watch?v=IVrGz8w0H8c](https://www.youtube.com/watch?v=IVrGz8w0H8c)*

2. Code Coverage with pytest-cov

Code coverage measures what percentage of your application code is actually exercised by your tests. A coverage report shows which lines were run (green) and which were never touched (red). High coverage (70-90%+) gives more confidence in your test suite.

```
pip install pytest-cov

# Run tests and show coverage in terminal
pytest --cov=.

# Generate a detailed HTML report
pytest --cov=. --cov-report=html

# Open the report in your browser
# Open: htmlcov/index.html

# Coverage for a specific module only
pytest --cov=src/login --cov-report=html

# Fail if coverage drops below a threshold
pytest --cov=. --cov-fail-under=80 # Fail if < 80% coverage
```

3. Test Ordering with pytest-ordering

By default pytest runs tests in the order they appear in the file. The **pytest-ordering** plugin lets you specify an explicit execution order. This is useful for tests that depend on each other (e.g., login must run before checkout).

```
pip install pytest-ordering

import pytest

@pytest.mark.run(order=1) # Runs FIRST
def test_login():
    print('Step 1: Login')

@pytest.mark.run(order=3) # Runs THIRD
def test_logout():
    print('Step 3: Logout')

@pytest.mark.run(order=2) # Runs SECOND
def test_add_to_cart():
    print('Step 2: Add to cart')

# Execution order: Login → Add to Cart → Logout
```

■ **TIP** Be careful with test ordering — tests should ideally be independent. If test 2 depends on test 1 passing, a failure in test 1 breaks your whole suite. Use fixtures and proper test setup instead of relying on order where possible.

■ **Pytest Ordering — Test Priority** | youtube.com/watch?v=cHYq1MRoyl0

WEEK 7 — Timeout, Parallel Testing & Parameterization

1. Test Timeouts with pytest-timeout

Sometimes tests hang indefinitely because a page never loads or an element never appears. The **pytest-timeout** plugin kills a test after a specified time limit, preventing your CI pipeline from blocking forever.

```
pip install pytest-timeout

import pytest

# Timeout this specific test after 10 seconds
@pytest.mark.timeout(10)
def test_slow_feature(page):
    page.goto('https://example.com/heavy-page')
    page.wait_for_selector('#main-content') # Must appear within 10 sec
    assert page.locator('#main-content').is_visible()

# Global timeout for ALL tests – add to pytest.ini:
# [pytest]
# timeout = 30

# Or pass on command line:
# pytest --timeout=30
```

■ **TIP** Set timeouts at the test level for tests with known slow pages, and a global timeout as a safety net in `pytest.ini`. Recommended global: 30-60 seconds.

■ *Pytest Timeout Plugin* | [youtube.com/watch?v=wVEiTPxVPy4](https://www.youtube.com/watch?v=wVEiTPxVPy4)

2. Parallel Testing with pytest-xdist

Running tests one at a time is slow for large test suites. **pytest-xdist** distributes tests across multiple CPU cores, running them simultaneously. A suite that takes 10 minutes sequentially might take just 2-3 minutes with 4 workers.

```
pip install pytest-xdist

pytest -n 2 # 2 parallel workers
pytest -n 4 # 4 parallel workers
pytest -n auto # Automatically use all available CPU cores

# Combine with verbose output
pytest -n 4 -v

# IMPORTANT: Tests running in parallel must be INDEPENDENT
# They cannot share state or write to the same file simultaneously

# Good practice: each test uses its own browser instance (via fixtures)
# @pytest.fixture ← function scope means each test gets its own browser
```

■ **TIP** Parallel testing requires tests to be independent. If test A modifies data that test B reads, you'll get unpredictable failures. Design each test to set up its own data and clean up after itself.

3. Parameterization — Data-Driven Tests

Parameterization runs the SAME test function with DIFFERENT sets of input data. Instead of writing 10 nearly identical test functions, write ONE function and provide 10 data sets. Pytest automatically creates a separate test for each row.

```
import pytest

# @pytest.mark.parametrize('param_names', [list_of_tuples])
@pytest.mark.parametrize('username, password, expected_msg', [
    ('standard_user', 'secret_sauce', 'Products'), # Valid login
    ('locked_out_user', 'secret_sauce', 'locked out'), # Locked account
    ('wrong_user', 'wrong_pass', 'Username and password'), # Wrong creds
])
def test_login(page, username, password, expected_msg):
    page.goto('https://www.saucedemo.com')
    page.locator('#user-name').fill(username)
    page.locator('#password').fill(password)
    page.locator('#login-button').click()
    assert expected_msg in page.inner_text('body')

# This creates 3 separate test cases automatically:
# test_login[standard_user-secret_sauce-Products]
# test_login[locked_out_user-secret_sauce-locked out]
# test_login[wrong_user-wrong_pass-Username and password]
```

■ **TIP** Combine parameterization with Excel reading (Week 4): read rows from a spreadsheet and pass them to parametrize. This is full Data-Driven Testing — test data is completely separate from test code.

WEEK 8 — Cross-Browser Testing, Reports, Allure & GitHub Copilot

1. Cross-Browser Testing with Parametrized Fixtures

One of Playwright's standout features is running the exact same tests across Chrome, Firefox, and Safari without changing a single line of test code. The trick is a parametrized fixture that provides a different browser per test run.

```
# conftest.py – cross-browser fixture
import pytest
from playwright.sync_api import sync_playwright

@pytest.fixture(params=['chromium', 'firefox', 'webkit'])
def page(request):
    with sync_playwright() as p:
        browser_type = getattr(p, request.param) # p.chromium / p.firefox / p.webkit
        browser = browser_type.launch(headless=False)
        page = browser.new_page()
        yield page
        browser.close()

# test_homepage.py – no changes needed!
def test_title(page):
    page.goto('https://www.saucedemo.com')
    assert page.title() == 'Swag Labs'

# This automatically runs test_title 3 times:
# test_title[chromium] ✓
# test_title[firefox] ✓
# test_title[webkit] ✓
```

2. Multi-Browser Parallel Testing

Combine cross-browser fixtures (3 browsers) with parallel execution (pytest-xdist) to run all browser tests simultaneously — maximum speed!

```
# Run 3 workers – one per browser – all simultaneously
pytest -n 3 -v

# Expected output:
# [chromium] test_login PASSED
# [firefox] test_login PASSED
# [webkit] test_login PASSED
# 3 passed in 12.5s (vs 35s sequentially)
```

3. HTML Reports with pytest-html

pytest-html generates a standalone HTML file with colour-coded pass/fail results, test duration, environment information, and — with the code below — screenshots automatically attached to every failed test.


```

pip install pytest-html

# Generate report
pytest --html=report.html --self-contained-html

# conftest.py – auto-screenshot on test failure
import pytest, os

@pytest.hookimpl(tryfirst=True, hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    rep = outcome.get_result()
    if rep.when == 'call' and rep.failed:
        page = item.funcargs.get('page')
        if page:
            os.makedirs('screenshots', exist_ok=True)
            path = f'screenshots/{item.name}.png'
            page.screenshot(path=path)
            print(f'Screenshot saved: {path}')

```

■ *pytest-html Report Generation | youtube.com/watch?v=7BJ_BKeeJyM*

4. Allure Reports — Beautiful Interactive Reports

Allure produces stunning interactive HTML reports with charts, timelines, step-by-step breakdowns, and history trends. It's the most widely used reporting tool in professional test automation.

```

pip install allure-pytest

# Run tests and save raw results
pytest --alluredir=./allure-results

# Open interactive report in your browser
allure serve allure-results

# Add Allure steps/labels to your tests for richer reports
import allure

@allure.title('Verify login with valid credentials')
@allure.description('This test verifies that a valid user can log in')
@allure.severity(allure.severity_level.CRITICAL)
def test_login(page):
    with allure.step('Open login page'):
        page.goto('https://www.saucedemo.com')
    with allure.step('Fill credentials'):
        page.locator('#user-name').fill('standard_user')
        page.locator('#password').fill('secret_sauce')
    with allure.step('Click login and verify'):
        page.locator('#login-button').click()
    assert 'inventory' in page.url

```

■ *Allure Report with Pytest — Playwright Python | youtube.com/watch?v=5gGpGVsF4XM*

5. GitHub Copilot — AI-Powered Code Assistant

GitHub Copilot is an AI pair programmer integrated into VS Code. It reads your code comments and context and suggests complete code snippets in real-time — powered by OpenAI's Codex model. It dramatically speeds up writing

boilerplate test code.

- **Install VS Code** from code.visualstudio.com
- **Install GitHub Copilot extension** from the VS Code Extensions panel
- **Sign in with GitHub** (requires a paid Copilot subscription)
- **Start typing a comment** — Copilot suggests the full implementation

```
# You type this comment:
# Write a Playwright test to login to saucedemo.com with valid credentials
# and verify the inventory page loads with at least 6 products

# Copilot suggests:
def test_login_and_verify_inventory(page):
    page.goto('https://www.saucedemo.com')
    page.locator('#user-name').fill('standard_user')
    page.locator('#password').fill('secret_sauce')
    page.locator('#login-button').click()
    assert page.url == 'https://www.saucedemo.com/inventory.html'
    items = page.locator('.inventory_item')
    assert items.count() >= 6
```

■ **TIP** Copilot use cases for testers: generate test cases from plain English comments, auto-complete CSS/XPath locators, explain unfamiliar code, suggest bug fixes, and convert manual test cases into automated scripts.

■ *GitHub Copilot Tutorial — Fireship | youtube.com/watch?v=Fi3AJZZregI*

QUICK REFERENCE — All Commands at a Glance

Install All Packages

```
pip install playwright pytest pytest-playwright
pip install pytest-html allure-pytest pytest-xdist
pip install pytest-timeout pytest-check pytest-ordering
pip install openpyxl pytest-cov
playwright install # Download browser binaries
```

Pytest Run Commands

Command	What it does
	Run all tests in current folder
pytest -v	Verbose — show each test name & result
	Also show print() output
pytest -n 4	Run 4 tests in parallel
	Run only tests with 'login' in the name
pytest --html=report.html	Generate HTML test report
	Save Allure results
pytest --cov=. --cov-report=html	HTML code coverage report
	Set 30-second global timeout
pytest test_login.py	Run one specific file
	Run one specific test function
pytest --tb=short	Shorter traceback on failures

Locator Quick Reference

Locator	Use When
	Element has accessible role + name
get_by_label('Email')	Input has a
	Input has placeholder text
get_by_text('Click here')	Element has unique visible text
	Element has a unique id attribute
locator('.classname')	Element has a unique class
	Element has a specific attribute

	Last resort — when nothing else works
--	---------------------------------------

Practice Recommendation

■ **TIP** Use <https://www.saucedemo.com> as your practice target — it has login, product listing, cart, and checkout flows. Cover all 8 weeks of concepts using just this one site!